JavaScript Getters and Setters

Introduction

Getters and setters are special accessor methods in JavaScript classes that provide controlled access to object properties. They allow you to define how properties are retrieved and modified, enabling data validation, computed values, and encapsulation.

Understanding the Fundamentals

The Distinction: Functions vs Variables

Before diving into getters and setters, it's essential to understand a core principle in object-oriented programming:

- **Variables (Properties)**: Store data
- **Functions (Methods)**: Perform work or operations

Getters and setters bridge these concepts by providing function-like behavior while maintaining property-like syntax.

Basic Class Structure

Let's examine a complete implementation of getters and setters using a Circle class:

```
class Circle {
    constructor(rad = 0) {
        this.rad = rad;
    }

    get radius() {
        return this.rad;
    }

    set radius(value) {
```


Powered by SPRK Technologies

```
        if (value > 0) {
            this.rad = value;
        } else {
            console.log("Radius must be above 0");
        }
    }

    get area() {
        return (Math.PI * this.rad ** 2).toFixed(2);
    }
}
```

Constructor Method

The `constructor(rad = 0)` initializes the circle with a radius value. The default parameter `rad = 0` ensures that if no value is provided during instantiation, the radius defaults to 0.

Getter Methods

Basic Getter: Retrieving Values

A getter method is defined using the `get` keyword followed by a method name. It allows you to access a property using simple dot notation without parentheses:

```
get radius() {
    return this.rad;
}
```

This getter returns the internal `rad` property. Notice that the internal property (`this.rad`) has a different name from the getter (`radius`). This naming convention prevents infinite recursion and clearly separates the internal state from the public interface.

Computed Getter: Dynamic Values

Getters can also perform calculations and return computed values:

Powered by SPRK Technologies

```
get area() {
    return (Math.PI * this.rad ** 2).toFixed(2);
}
```

This getter calculates the circle's area using the formula πr^2. The `toFixed(2)` method formats the result to two decimal places. Importantly, this computation happens every time the property is accessed, ensuring the value is always current.

Setter Methods

Validation and Data Integrity

A setter method is defined using the `set` keyword and accepts exactly one parameter. Setters enable validation logic before modifying internal state:

```
set radius(value) {
    if (value > 0) {
        this.rad = value;
    } else {
        console.log("Radius must be above 0");
    }
}
```

This setter validates that the radius value is positive before assignment. If the validation fails, it logs an error message and prevents the invalid state change. This demonstrates a key principle: setters enforce business rules and maintain data integrity.

Practical Usage

Creating and Using Instances

```
const c = new Circle(4.5);
console.log(c.radius);  // Accessing via getter
console.log(c.area);    // Accessing computed property
```


Powered by SPRK Technologies

```
c.radius = 34;          // Modifying via setter
console.log(c.area);    // Area automatically reflects new radius
```

Output:

```
4.5
63.62
3631.68
```

Attempting Invalid Assignment

```
c.radius = -5;          // Triggers validation in setter
console.log(c.radius);  // Value remains unchanged
```

Output:

```
Radius must be above 0
34
```

Key Advantages

1. Encapsulation

Getters and setters allow you to hide the internal representation of data. The external interface (`radius`) remains consistent even if the internal implementation (`rad`) changes.

2. Validation

Setters provide a centralized location for validation logic, preventing invalid states and reducing bugs throughout your application.

Powered by SPRK Technologies

3. Computed Properties

Getters enable properties that are calculated on-the-fly, avoiding the need to store redundant data and ensuring values are always synchronized.

4. Clean Syntax

Despite being methods, getters and setters are accessed like regular properties, providing an intuitive API for developers using your classes.

Important Considerations

Read-Only Properties

A getter without a corresponding setter creates a read-only property. In our example, `area` can only be read, not set directly:

```
console.log(c.area);    // ✓ Works
c.area = 100;           // ✗ Silently fails (or throws in strict mode)
```

Output:

```
3631.68
```

Naming Conventions

When using getters and setters, the internal property typically uses a different name (often prefixed with an underscore or abbreviated). This prevents naming conflicts and infinite recursion.

Performance

Getters execute every time the property is accessed. For expensive computations, consider caching the result if the underlying data hasn't changed.

Powered by SPRK Technologies

Complete Working Example

```
class Circle {
    constructor(rad = 0) {
        this.rad = rad;
    }

    get radius() {
        return this.rad;
    }

    set radius(value) {
        if (value > 0) {
            this.rad = value;
        } else {
            console.log("Radius must be above 0");
        }
    }

    get area() {
        return (Math.PI * this.rad ** 2).toFixed(2);
    }

    get circumference() {
        return (2 * Math.PI * this.rad).toFixed(2);
    }
}

// Usage
const circle = new Circle(10);
console.log(`Radius: ${circle.radius}`);
console.log(`Area: ${circle.area}`);
console.log(`Circumference: ${circle.circumference}`);

circle.radius = 15;
console.log(`New Area: ${circle.area}`);
console.log(`New Circumference: ${circle.circumference}`);
```

Output:

Powered by SPRK Technologies

```
Radius: 10
Area: 314.16
Circumference: 62.83
New Area: 706.86
New Circumference: 94.25
```

Summary

Getters and setters are powerful features in JavaScript that enable controlled property access. They provide validation, enable computed properties, and maintain clean, intuitive APIs. By understanding when and how to use these accessor methods, you can write more robust, maintainable object-oriented JavaScript code.

Powered by SPRK Technologies